

Personal Finance Dashboard

Architecture Documentation (arc42)

Roman Lysser · Module WEBLAB.F2601 · 25. September 2026

Contents

1. Introduction and Goals	3
1.1. Requirements Overview	3
1.2. Quality Goals	3
1.3. Stakeholders	3
2. Architecture Constraints	4
3. System Scope and Context	5
3.1. Business Context	5
3.2. Technical Context	5
4. Solution Strategy	6
4.1. Technology and decomposition	6
4.2. Quality goals	6
5. Building Block View	7
5.1. Level 1: Whitebox Overall System	7
5.2. Level 2: Backend Architecture	8
5.3. Level 2: Frontend Architecture	9
6. Runtime View	11
6.1. Sign-in and session	11
6.2. Onboarding	12
6.3. Monthly limits take-over	12
7. Deployment View	13
7.1. Runtime configuration	14
7.2. Seed users	14
8. Cross-Cutting Concepts	15
8.1. Domain model	15
8.2. Household scoping	15
8.3. Domain errors at the HTTP boundary	15
8.4. Money and account numbers	15
8.5. Security	16
8.6. Theming	16
8.7. Responsive shell and dialogs	16
8.8. Quick transaction entry	16
8.9. Route loading	17
8.10. Testing	17
9. Architecture Decisions (ADRs)	19
9.1. ADR-1: "Uncategorized" is the absence of a category, not a reserved category row	19

9.2.	ADR-2: A new month takes over the nearest earlier limits, automatically only for the current and next month	19
9.3.	ADR-3: Use libsql batch() for atomic multi-statement writes	22
9.4.	ADR-4: Mount better-auth's HTTP handler before Nest initializes	22
9.5.	ADR-5: Real sessions replace the simulated login; onboarding creates the household	23
9.6.	ADR-6: Require BETTER_AUTH_SECRET instead of shipping a default	23
10.	Quality Requirements	25
10.1.	Quality Tree	25
10.2.	Quality Scenarios	25
10.3.	Lighthouse Results	26
11.	Risks and Technical Debt	28
12.	Glossary	29

1. Introduction and Goals

1.1. Requirements Overview

The dashboard lets a household record accounts and transactions, plan monthly limits per category, and see balances and spending. Story IDs such as M5 or S4 refer to docs/proposal.typ.

1.2. Quality Goals

Priority	Goal	Why
1	Atomic financial writes	A failed multi-step write leaves no partial household, transfer, or history.
2	Responsive navigation	The first page loads quickly on slow connections, and the layout works on phone and desktop.
3	Low-effort transaction entry	If entering is tedious, entries get skipped and balances and limits drift from reality.

1.3. Stakeholders

Role	Expectation
Developer (Roman Lysser)	Well-thought-out architecture, tools that are clear to use, fast-running scripts, and a polished overall look and feel.
Lecturer (HSLU)	Minimum project requirements: • Create, show, change, and delete a self-defined resource, stored in a database and shown in at least two different views. • Responsive for desktop, tablet, and mobile. • Automated unit, integration, and end-to-end tests. • Lighthouse scores average 90 or more on mobile and desktop. • Reproducible start via a public URL or one docker compose up. • Readable, extensible, well-structured code.
Household member (end user)	A well-structured overview, ease of use, and reliability.

2. Architecture Constraints

Constraint	Background
TypeScript, Angular 22, NestJS 12	Stack fixed in the proposal; one language from database to view.
SQLite, no extra infrastructure	One database file via libsql; runs with docker compose up only.
No bank connection	Switzerland has no bank-agnostic API for reading balances and transactions, so transactions are entered by hand; balance = opening balance + transactions.
One currency per household	household.base_currency (CHF, EUR, USD, GBP); no exchange rates.
Single developer, course timeline	Module WEBLAB.F2601; scope bounded by the MoSCoW stories of the proposal and the lecturer's minimum requirements (ch. 1 Stakeholders).

3. System Scope and Context

3.1. Business Context

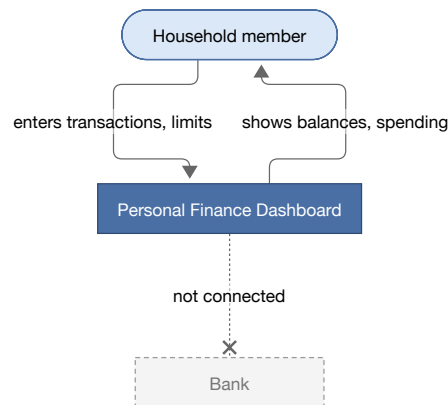


Figure 1: The household member is the only partner. No bank is connected (ch. 2).

Partner	Input	Output
Household member	Household setup, accounts, transactions, monthly limits	Balances, transaction history, category statistics, budget status

3.2. Technical Context

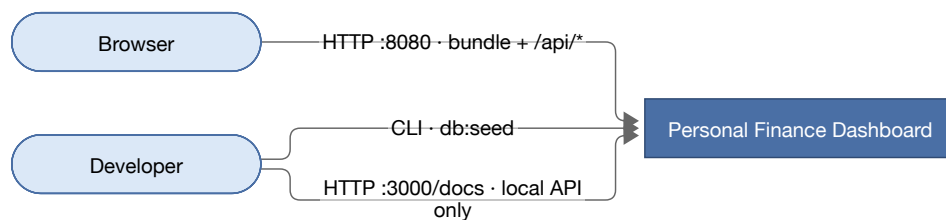


Figure 2: Users reach the Compose stack on port 8080; Swagger exists only on a locally started API.

Partner	Channel	Details
Browser	HTTP	Compose stack: one origin on port 8080 serves the app bundle and the REST API under /api/* (ch. 7).
Developer	CLI	bun run db:seed, bun run openapi:generate.
Developer	HTTP	Swagger UI at http://localhost:3000/docs when the API runs locally; not in the Compose stack (ch. 8 Security).

4. Solution Strategy

4.1. Technology and decomposition

Decision	Reason (alternative beaten)
SPA + REST API + SQLite	Browser app, one API process, and one database file (ch. 5) cover a household-sized load; a file database needs no server (beat PostgreSQL).
Modular monolith by feature	backend/src/features/* and frontend/src/app/features/* mirror the domain; the backend stays one deployable API (beat type folders such as controllers/, services/).
Drizzle ORM + drizzle-kit migrations	Typed schema next to its feature; SQL-close queries (beat TypeORM decorators).
better-auth sessions	Cookie sessions and rate limiting out of the box, tables in the same Drizzle schema (beat a hand-written JWT flow). Mounted before Nest's body parser (ADR-4); replaces the simulated login (ADR-5).
OpenAPI → generated client	@nestjs/swagger spec generates frontend/src/app/core/api/; no duplicated DTOs.
Angular Material + Tailwind	Accessible components plus semantic color tokens (ch. 8 Theming).
Monorepo, Bun	One repository for frontend/ and backend/; Bun as package manager and runtime.

4.2. Quality goals

Quality goal	Approach
Atomic financial writes	Multi-statement writes run as one libsql batch() (ADR-3).
Responsive navigation	Prefetch the current tab at boot, preload the rest later; one scroll panel in a fixed shell (ch. 8 Route loading, Responsive shell).
Low-effort transaction entry	One add button; the dialog needs only the amount, everything else is prefilled or optional (ch. 8 Quick transaction entry).

5. Building Block View

5.1. Level 1: Whitebox Overall System

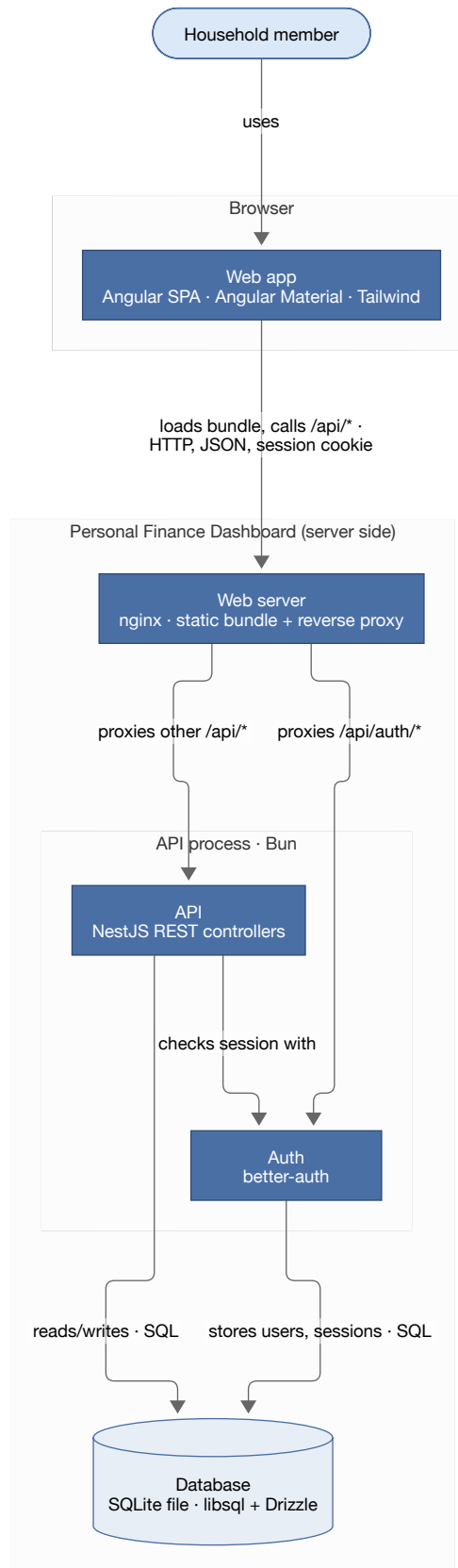


Figure 3: One browser app, one reverse proxy, one API process with auth inside, one SQLite file.

Building block	Responsibility	Source
Web app	Screens, client state, routing, theming	frontend/
Web server	Serves the bundle, proxies /api/	frontend/nginx.conf.template
API	REST endpoints, business rules, household scoping	backend/src/features/
Auth	Sign-in, session cookies, rate limiting	backend/src/shared/infra/auth/
Database	Persistent state; schema via migrations	backend/drizzle/

5.2. Level 2: Backend Architecture

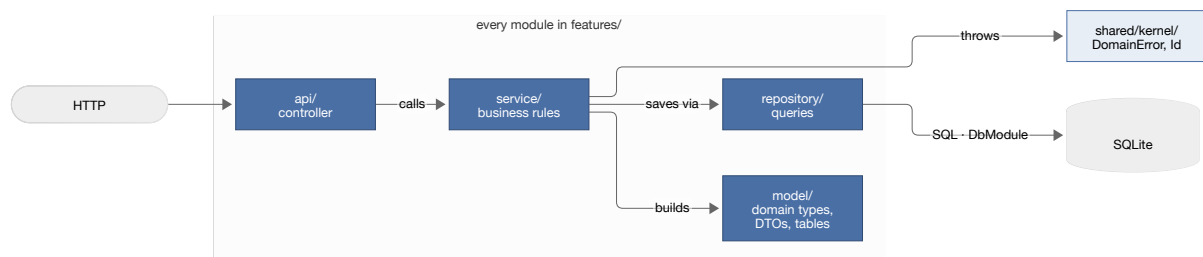


Figure 4: Every feature module has the same four layers: controllers never touch the database, services never see HTTP.

Module	Responsibility
household	Membership and household settings; provides HouseholdMemberGuard (ch. 8 Security).
onboarding	Wizard validation and atomic household creation (ADR-3).
account	Finance accounts: numbering, archive or delete (M16).
category	Categories, default list, transfer on delete (ADR-1).
transaction	Transactions, filters, statistics.
budget	Monthly limits and take-over (ADR-2); checks categories via category.
shared/infra/	AuthModule (global AuthGuard), DbModule (libsql), HealthModule, setupApp() (CORS, auth handler, validation, error filter; ADR-4).
shared/kernel/	Id, Currency, DomainError; no Nest or Drizzle imports.

New tables must be re-exported from shared/infra/db/schema.ts: drizzle-kit and the better-auth adapter both read the schema from there.

5.3. Level 2: Frontend Architecture

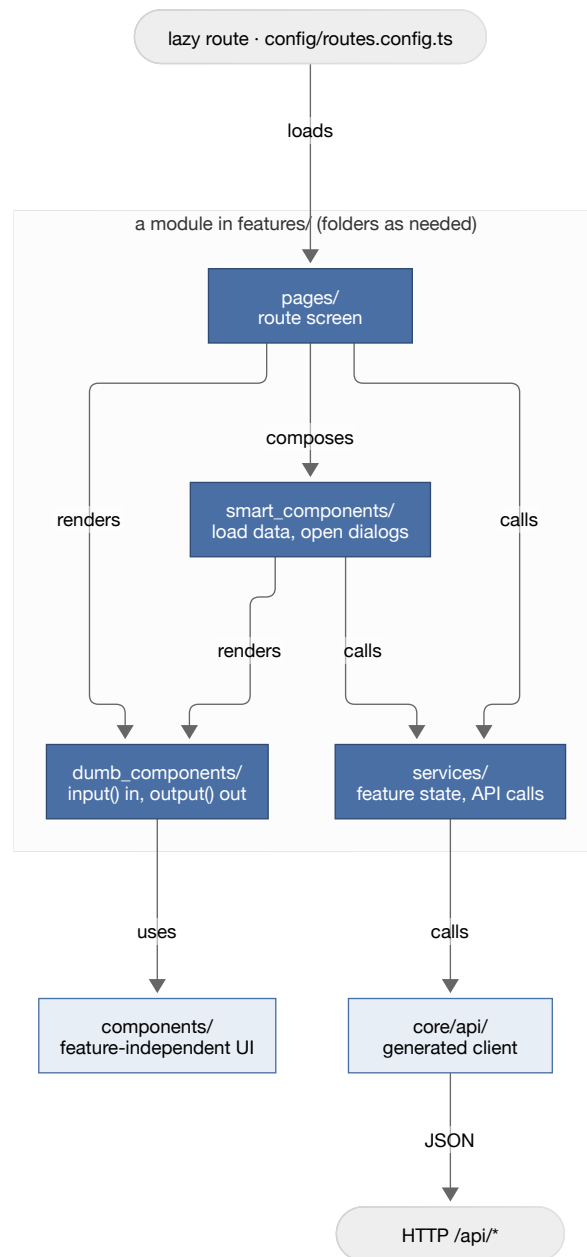


Figure 5: Feature modules use these folders as needed: *budget* and *transaction* have all four, *shell* and *settings* only pages. Dumb components only get inputs and emit outputs; only services call the API.

Module	Responsibility
shell	Authenticated layout, tab navigation, the single scroll panel (ch. 8 Responsive shell).
auth	Login page and form.
onboarding	Wizard, localStorage draft, validation calls, final submit.
household	Home page; HouseholdService and the onboarding guards.
transaction	Transactions page, history, add/edit dialog, last-used defaults (ch. 8 Quick transaction entry).

Module	Responsibility
analytics	Analytics tab layout and category statistics chart.
budget	Budgets page, limit dialog, safe-to-spend card, take-over (ADR-2).
settings	Household settings, account and category management, sign-out.
account, category, stats	No own page: components and services reused by Home and Settings.
config/, core/, components/	Shared: routes and preloading (ch. 8 Route loading), generated API client, auth client and guard, theme, i18n, and feature-independent UI such as the dialog service.

Features may import other features' components and services (Home shows account, budget, and stats components); shared code never imports from a feature (`frontend/AGENTS.md`).

6. Runtime View

6.1. Sign-in and session

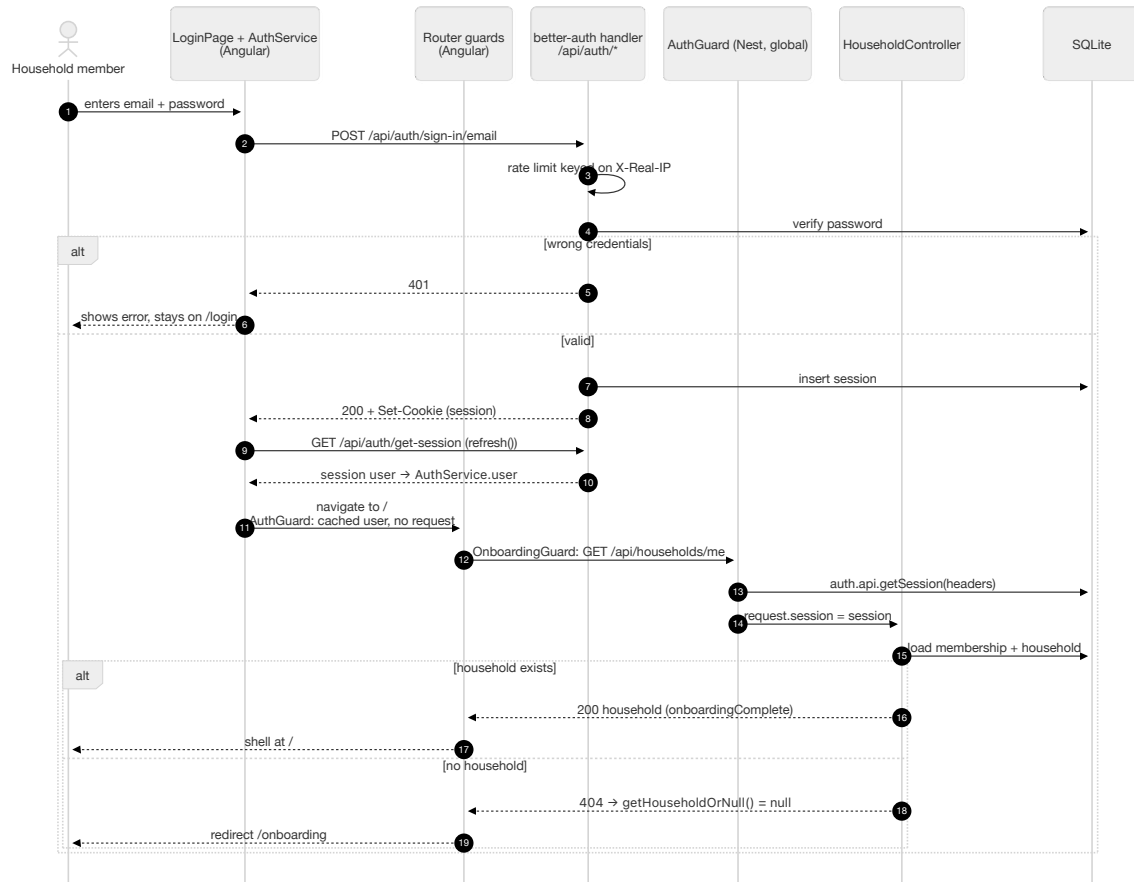


Figure 6: The session lives in an HTTP-only cookie; the SPA never handles a token.

- Router guards call GET /api/households/me: 404 means no household yet and redirects to / onboarding; otherwise the shell opens.

6.2. Onboarding

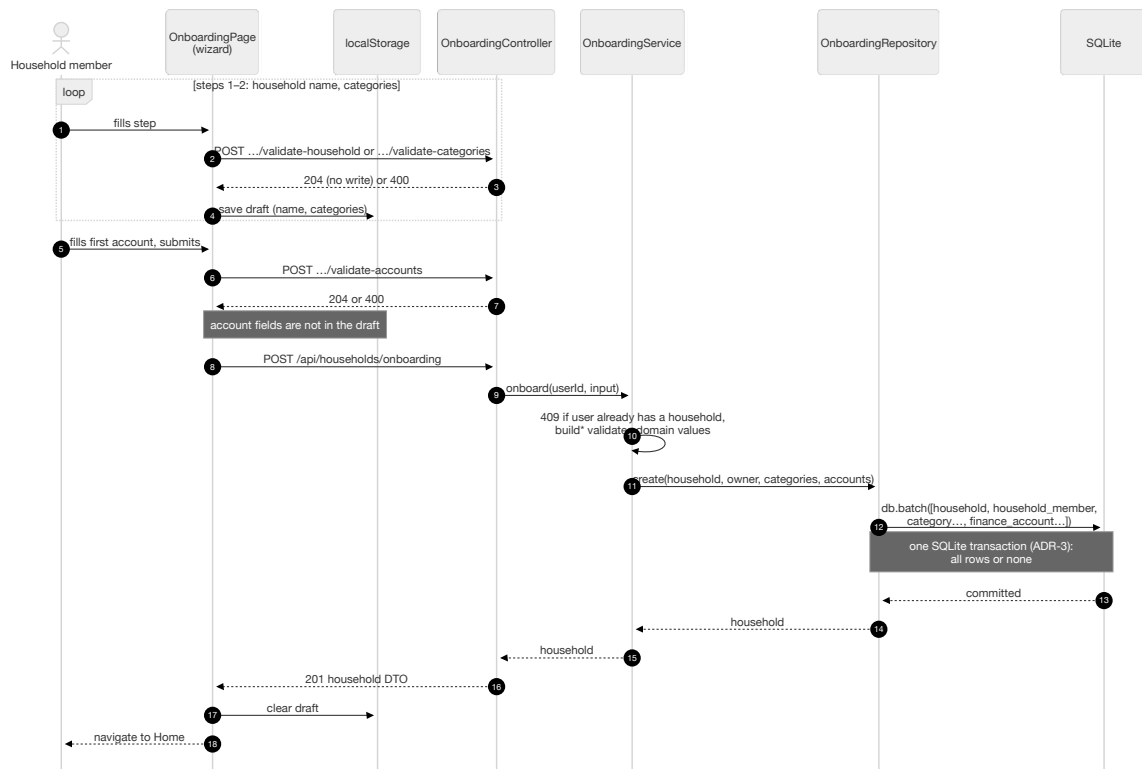


Figure 7: Validation steps never write; the single submit creates the whole household in one libsql batch.

- A page reload resumes the wizard from the localStorage draft, which holds the household name and categories; account fields must be entered again. Only the final submit writes (ADR-3).

6.3. Monthly limits take-over



Figure 8: Opening an empty current or next month copies the nearest earlier limits once; a month the user emptied stays empty. Rules and reasons: ADR-2.

- BudgetService.loadMonth (frontend) runs the take-over before Home or the budgets tab renders, so both show the same limits.

7. Deployment View

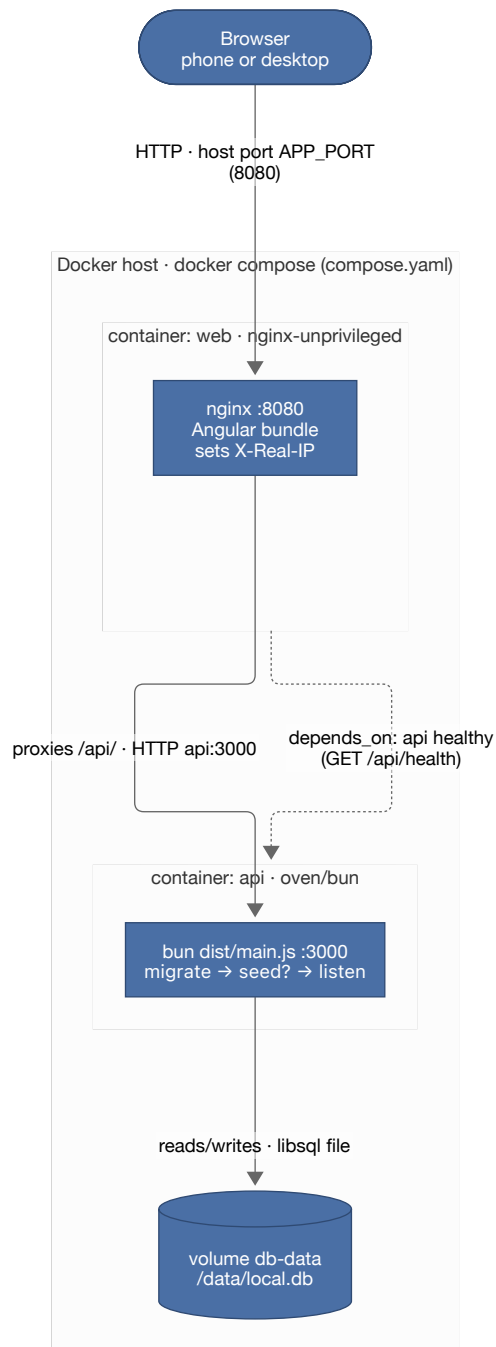


Figure 9: Only nginx is reachable from outside; the API starts after migrations and is healthy before nginx starts.

Node	Artifact and configuration
web	Angular browser bundle in nginx. Host port APP_PORT defaults to 8080; /api/ proxies to API_UPSTREAM=api:3000.
api	Bun/NestJS image, port 3000. DB_FILE_NAME=file:/data/local.db; migrations run at startup before listening.
db-data	Named Docker volume holding the SQLite file across container restarts.

7.1. Runtime configuration

Variable	Required	Purpose
BETTER_AUTH_SECRET	Compose, prod	Signs session cookies; no default value (ADR-6).
BETTER_AUTH_URL TRUSTED_ORIGINS	Other origin	Must match the browser-facing origin when it is not <code>http://localhost:8080</code> .
SEED_DEMO	No	Creates the seed users below on startup. Default <code>true</code> in Compose, <code>false</code> locally; disable outside demos.
APP_PORT	No	Published HTTP port for <code>web</code> ; defaults to 8080. TLS termination is outside this stack (ch. 11).

7.2. Seed users

- `demo@finance.local` / `demo-password`: no household yet, starts the onboarding wizard.
- `sample@finance.local` / `sample-password`: household with categories, accounts, budgets, and 200 transactions.
- `SEED_DEMO` only creates missing users and never resets data. `bun run db:seed` (local development) resets both households (`backend/src/shared/infra/db/seed.ts`).

8. Cross-Cutting Concepts

8.1. Domain model

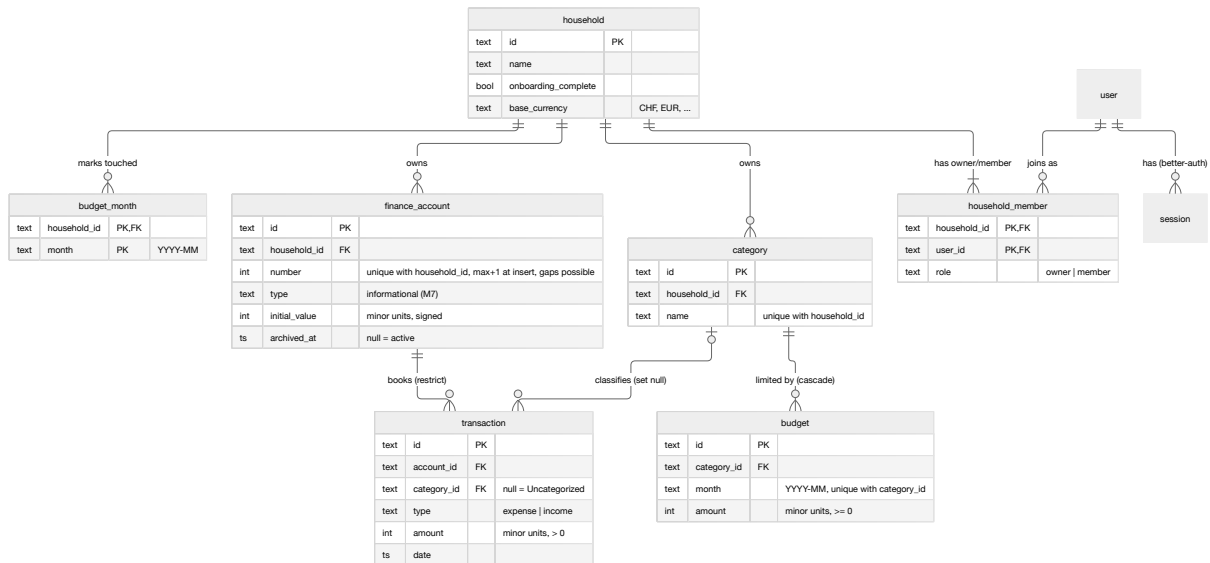


Figure 10: Household is the ownership root; transaction and budget reach it only through their parents. user and session belong to better-auth.

- `transaction.category_id` null means “Uncategorized” (ADR-1).
- The table is `finance_account` (Drizzle symbol `financeAccount`) because better-auth already uses account for login providers.

8.2. Household scoping

- `finance_account`, `category`, and `budget_month` carry `household_id`; `transaction` and `budget` inherit ownership through their parents, without redundant household columns.
- Child reads follow the parent: `TransactionRepository.inHousehold` scopes through the account; `budget` reads join `category` and filter `category.householdId`.
- Services check parent ownership before child writes.

8.3. Domain errors at the HTTP boundary

Services throw HTTP-independent `DomainError` subclasses (`backend/src/shared/kernel/domain-error.ts`). `DomainExceptionFilter` maps them at the API boundary: `not_found 404` · `forbidden 403` · `conflict 409` · `validation 400`. Responses contain `statusCode`, the error class name, and its message; domain operations remain usable outside HTTP.

8.4. Money and account numbers

- API and database amounts are integer minor units (cents for supported currencies); UI inputs round major units on submission and divide by 100 for display.
- Opening balances may be signed; transaction amounts are positive with direction in type; budget limits are nonnegative.
- Input DTOs cap a single absolute amount at `MAX_AMOUNT = 100_000_000_000` minor units (one billion major units), leaving headroom below JavaScript’s `Number.MAX_SAFE_INTEGER` for balance sums. (`backend/src/shared/kernel/currency.ts`).
- Account numbers are unique per household. `nextAccountNumber` computes `max + 1` inside the insert statement, so numbers stay correct inside a batch and under concurrent requests (ADR-3, `account.schema.ts`).

8.5. Security

- better-auth owns sign-in and session cookies (ADR-4). AuthModule installs a global guard: every controller route needs a session unless marked `@Public()`, and household routes also pass `HouseholdMemberGuard`.
- `setupApp` allows credentialed CORS only for `TRUSTED_ORIGINS`.
- `nginx` is the Compose entry point and overwrites `X-Real-IP` with `$remote_addr`; better-auth rate limits on that header instead of client-controlled `X-Forwarded-For`. With an upstream proxy, `$remote_addr` is that proxy until trusted real-IP handling is configured (`frontend/nginx.conf.template`).
- Production requires `BETTER_AUTH_SECRET` to prevent forgeable session cookies (ADR-6); Swagger `/docs` and OpenAPI generation run only outside production, so never in the Compose stack (`backend/src/shared/infra/config/env.ts`, `backend/src/main.ts`).

8.6. Theming

- Material 3 `--mat-sys-*` color/type tokens and app `--app-*` tokens (income, expense, warning, accounts) map to semantic Tailwind names such as `bg-surface-low` and `text-income`; templates never use palette utilities.
- Material and app colors use `light-dark()`; `ThemeService` stores light, dark, or follow-system preference per browser and sets `<body>`'s `color-scheme`.
- Canvas cannot resolve `light-dark()`: charts use `ThemeService`'s resolved scheme and resolved Material ink/border colors; CSS avatars can use `light-dark()` directly.

8.7. Responsive shell and dialogs

- `ShellPage` fills `h-dvh`; `MatTabNavPanel` is the sole scroll container and the document disables overscroll. Bottom navigation on phones becomes header tabs at `md`; the panel keeps content above iOS Firefox's translucent bottom toolbar (M15).
- `.app-dialog` is a full-screen sheet below 640px and a centered card above it, including when resized while open.
- Before a lazy dialog import, `DialogService.open` focuses an off-screen input within the tap so iOS keeps the keyboard open when CDK moves focus to `cdkFocusInitial`. Editing may use `focusInput: false`; focus returns to the trigger on close.

8.8. Quick transaction entry

A repeat entry needs only the amount (quality goal 3).

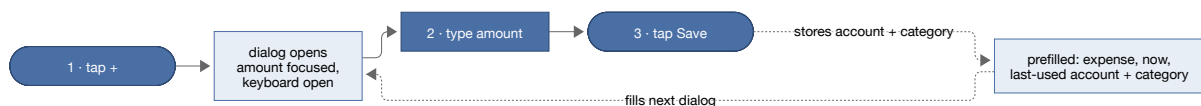


Figure 11: Tap, amount, Save. Everything else is prefilled from the last entry.

- Category and title are optional (ADR-1).
- Code: `TransactionHistoryComponent` (add button), `TransactionFormComponent` (defaults), `TransactionService` (localStorage key `transaction-last-used`).

8.9. Route loading

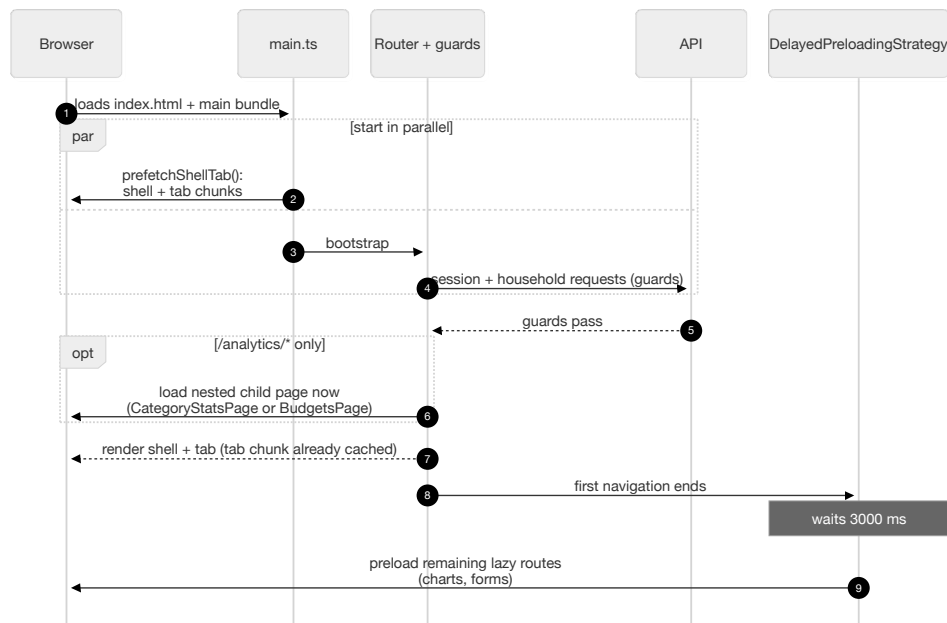


Figure 12: The shell and the tab page download in parallel with the guard requests; everything else waits 3 s.

For `/analytics/*`, only the `AnalyticsPage` layout is prefetched; its nested child loads after the guards pass. Other lazy routes preload 3 s after the first navigation completes, so chart and form chunks do not compete with the first page on slow connections. The delay does not wait for the page's own data requests (QS-2, ch. 10).

8.10. Testing

Four suites cover unit, integration, and end-to-end level. All passed on 2026-09-25:

Suite	Tool	Files	Tests	What is tested
Backend unit	Vitest	24	191	Model rules (build*: amounts, months, date ranges, unique names), services, controllers and mappers, guards, env config, <code>DomainExceptionFilter</code> .
Backend integration	Vitest, supertest	10	111	HTTP contract of every feature: auth, onboarding, accounts, categories, transactions and filters, statistics, budgets; household isolation (foreign household's URL or ids); atomic budget writes (fault injection).
Frontend unit	Vitest, jsdom	53	277	Services (API calls, onboarding draft, last-used defaults), dumb components (inputs, outputs, forms), smart components and dialogs, guards, theme, i18n, preload strategy.

Suite	Tool	Files	Tests	What is tested
Browser end-to-end	Playwright	6	8 × 2	Login (anonymous redirect, wrong password), onboarding, account create and rename, transaction add/edit/delete with balance, income and expense on Home, budget overspend; each flow on desktop and mobile.

Coverage (V8 provider, 2026-09-25):

Suite	Statements	Branches	Functions	Lines
Backend unit	76.2 %	72.1 %	54.0 %	76.4 %
Backend integration	89.3 %	69.9 %	87.2 %	89.9 %
Frontend unit	87.4 %	79.7 %	84.2 %	87.0 %

Backend unit specs fake repositories, so repositories, Drizzle schemas, and DTOs count as 0 % there; the integration suite covers them. Browser end-to-end runs are not measured.

- **Where:** backend `src/**/*.spec.ts` (`bun run test`) and `test/*.e2e-spec.ts` (`bun run test:e2e`); frontend `src/**/*.spec.ts` (`bun run test`) and `playwright/e2e/` (`bun run e2e`). Coverage: `bun run test:cov` in both projects, `bun run test:e2e:cov` in the backend.
- **Fakes over mocks:** unit specs replace collaborators with small fakes typed as `Pick<Repository, 'method'>` or `Pick<Service, 'method'>`, implementing only the methods the subject calls; a cast supplies the full class type.
- **No mocks in integration:** backend integration specs run the real `AppModule`, including `setupApp()` and `better-auth`, on `:memory:` SQLite migrated and seeded per run (`test/setup-db.ts`).
- **Isolated browser flows:** Playwright starts its own API (`:3100, :memory:`) and `ng serve (:4300)`, in projects `desktop` (Desktop Chrome) and `mobile` (Pixel 7, QS-4). Fixtures in `playwright/fixtures/test.ts` sign up a fresh user, and onboard a household when needed, through the API for each test, so tests share no state and run fully parallel.

9. Architecture Decisions (ADRs)

9.1. ADR-1: “Uncategorized” is the absence of a category, not a reserved category row

Status: accepted · deviates from the wording of M5 and M6 in the proposal.

9.1.1. Context

M5 lets a transaction be entered without choosing a category, M6 lets a category be deleted without leaving its transactions unassigned. The proposal describes both with a *reserved catch-all category* that such transactions fall into.

9.1.2. Options

Option	Trade-off
Reserved category row	Must be seeded per household, protected from rename/deletion/self-transfer, omitted from onboarding suggestions, and identified by a flag or magic name in special-case queries.
Nullable category_id	Database handles deletion with <code>on delete set null</code> ; the bucket cannot be renamed or given its own limit.

9.1.3. Decision

- `transaction.category_id` is nullable; `null` means “Uncategorized”. Deleting a category sets its transactions to `null` (`on delete set null`) unless the user transfers them to another category.
- The transaction list, history filter (`categoryId=none`), statistics, and chart show “Uncategorized” as a bucket of its own.

9.1.4. Consequences

- M5/M6 hold: entry needs no category, and deletion offers another category or “Uncategorized”; every transaction has a visible bucket. The database fallback needs no row to seed, protect, migrate, or hide.
- The label comes from the translation files and cannot be renamed. Limits use (`category_id`, `month`), so it has no limit; its spending still reduces monthly cashflow and Safe-to-Spend. No Should-have story requires that limit.
- If a limit becomes necessary, insert a reserved category per household and move the `null` rows to it.

9.2. ADR-2: A new month takes over the nearest earlier limits, automatically only for the current and next month

Status: accepted · deviates from the wording of S4 in the proposal.

9.2.1. Context

S4 asks that a new month takes over the limits of the *previous month*, can be changed there, and leaves the previous month unchanged, so that limits need not be entered again every month.

Three questions are left open by that sentence:

- What counts as the previous month when that month has no limits, for example because the app was not used in August?
- When is a month “new”? The system has no scheduler, and a month nobody opens needs no limits.

- What happens to a month the user emptied on purpose? S1 lets every limit be removed, S3 lets a month stay deliberately unbudgeted. A rule that refills every empty month undoes both.

9.2.2. Decision

- Copy independent budget rows from the nearest earlier month with limits; reads never inherit limits, so copies can change without changing their source and “no row” still means “no limit” (S3).
- On first view, POST .../:month/copy-previous?auto=true applies only to the current and next month. Sequence: ch. 6 Monthly limits take-over. Other months need an explicit action; a month that already has limits returns 409.
- budget_month(household_id, month) marks months whose limits were set, removed, or taken over, in the same batch as the change (ADR-3). Automatic take-over skips touched months; the explicit action may refill them.

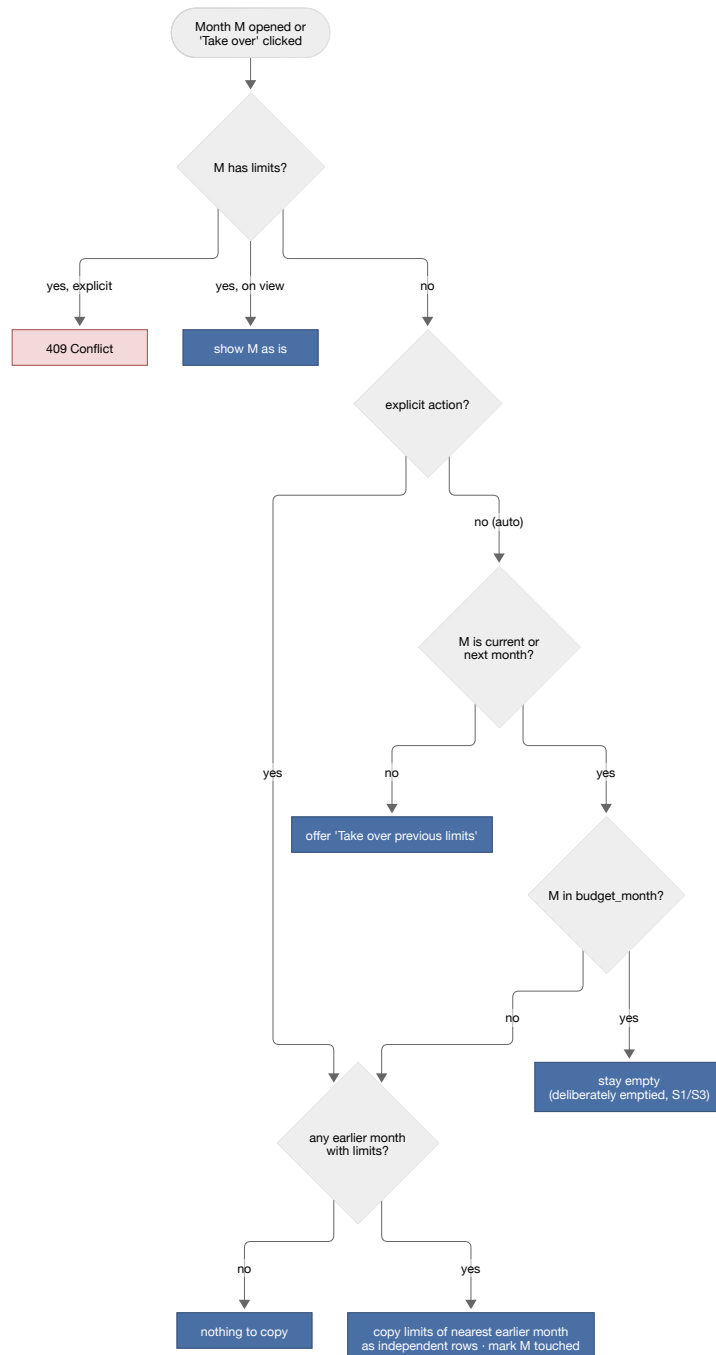


Figure 13: Take-over decision as implemented in *BudgetService.copyFromPrevious* (backend) and *loadMonth* (frontend).

9.2.3. Consequences

- S1, S3, and S4 hold together: a new month shows earlier limits, editing them leaves the source month unchanged, and a month the user emptied stays empty.
- Unopened months cost nothing; past months fill only on request.
- Skipping is decided per month: an emptied August does not stop September from taking over July's limits. Treating an emptied month as "inherit nothing" was rejected because it would lose the plan after every unused month.
- The trigger is the client clock (current/next month), since the server knows no household timezone. A client with a wrong clock can at most fill a month early; the user can remove the copies.

9.3. ADR-3: Use `libsql batch()` for atomic multi-statement writes

Status: accepted.

9.3.1. Context

Onboarding inserts a household, owner membership, categories, and accounts together. Category transfer updates transactions before deleting the category; a household currency change updates accounts with the household; every limit change also marks its month (ADR-2). In the `libsql` in-memory e2e database, `db.transaction` swaps the client's connection after an explicit transaction and loses that database.

9.3.2. Options

Option	Trade-off
<code>db.transaction</code>	Natural Drizzle API, but the <code>libsql</code> connection swap breaks <code>:memory:</code> e2e state.
Sequential writes	Simple, but a later failure can leave a partial household or transfer.
<code>db.batch</code>	<code>libsql</code> executes the statements in one transaction and rolls back together; statements must be assembled before execution.

9.3.3. Decision

Use `db.batch()` in `OnboardingRepository`, `CategoryRepository.deleteCategory` when transferring, `HouseholdRepository.update` when changing currency, and the `BudgetRepository` writes (`upsert`, `delete`, `insertMany`) together with the month marker.

9.3.4. Consequences

- These writes remain atomic in production and against the in-memory e2e database.
- Operations that need a result to choose the next statement cannot use this prepared batch shape; they need a separate design.
- Revisit if `libsql` fixes explicit-transaction connection handling or the persistence driver changes.

9.4. ADR-4: Mount `better-auth`'s HTTP handler before Nest initializes

Status: accepted.

9.4.1. Context

`better-auth`'s Node handler needs the raw request body for `/api/auth/*`. Nest's default body parser consumes that stream during initialization. Auth endpoints also need CORS headers even when the handler responds without entering Nest's controller pipeline.

9.4.2. Options

Option	Trade-off
Nest controller wrapper	Would run after body parsing and hand the handler a consumed stream.
Disable Nest body parsing	Preserves the auth stream but requires manual parser wiring for ordinary controllers.
Early Express middleware	Keeps Nest defaults for controllers while giving <code>better-auth</code> the raw stream.

9.4.3. Decision

setupApp registers CORS, then mountAuthHandler, before app.init() or listen(). The middleware handles /api/auth/* directly and calls next() for other paths. AuthModule remains inside Nest to expose the auth instance and install the global controller guard.

9.4.4. Consequences

- Requests take two paths: better-auth answers /api/auth/* and owns validation and responses there; all other requests pass Nest's body parser, guards, validation pipe, and error filter.
- The order in app.setup.ts and auth.handler.ts is part of the contract and must stay beside the code.
- Revisit if Nest or better-auth gains an integration that preserves the raw stream and response behavior.

9.5. ADR-5: Real sessions replace the simulated login; onboarding creates the household

Status: accepted · deviates from M1 and C1 in the proposal.

9.5.1. Context

M1 asks for a simulated login with a fixed household until real authentication exists. C1 (Could have) asks for a sign-up that creates the household with the user as owner. A usable household also needs a name, categories, and a first account (M4, M7).

9.5.2. Options

Option	Trade-off
Simulated login (M1)	No auth code, but every request trusts one fixed user; must be replaced later.
Sign-up creates the household (C1)	One step, but the household starts without categories or accounts.
Real sessions + onboarding	better-auth from the start; one request creates a complete household.

9.5.3. Decision

- better-auth sessions from the start (ADR-4). Signing up creates only the user.
- POST /api/households/onboarding creates the household with the user as owner, together with categories and the first account (ADR-3).
- The frontend has a login page but no sign-up page; users come from the seed (ch. 7) or POST /api/auth/sign-up/email.

9.5.4. Consequences

- M1 is exceeded: real access control instead of a simulation.
- C1 is partly met: sign-up works through the API only, and the household comes from onboarding, not from sign-up.
- A sign-up page needs frontend work only; after sign-up, the onboarding guard takes over.

9.6. ADR-6: Require BETTER_AUTH_SECRET instead of shipping a default

Status: accepted · qualifies M20 in the proposal.

9.6.1. Context

M20 asks that `docker compose up` starts the production bundle without extra knowledge. `better-auth` signs session cookies with `BETTER_AUTH_SECRET`; anyone who knows the secret can forge sessions.

9.6.2. Options

Option	Trade-off
Default secret in <code>compose.yaml</code>	True one-command start, but every deployment that forgets to override it has forgeable sessions.
Generate on first start	One command, but the secret must be persisted, or every restart signs everyone out.
Require it in <code>.env</code>	One setup step before <code>docker compose up</code> ; no known secret ever ships.

9.6.3. Decision

Compose refuses to start without `BETTER_AUTH_SECRET` (`${BETTER_AUTH_SECRET:?...}`), and `env.ts` requires it when `NODE_ENV=production`. `.env.example` documents the variable.

9.6.4. Consequences

- M20 holds with one setup step: `cp .env.example .env` and set the secret (QS-6).
- Revisit if the stack gets a persistent secret store.

10. Quality Requirements

10.1. Quality Tree

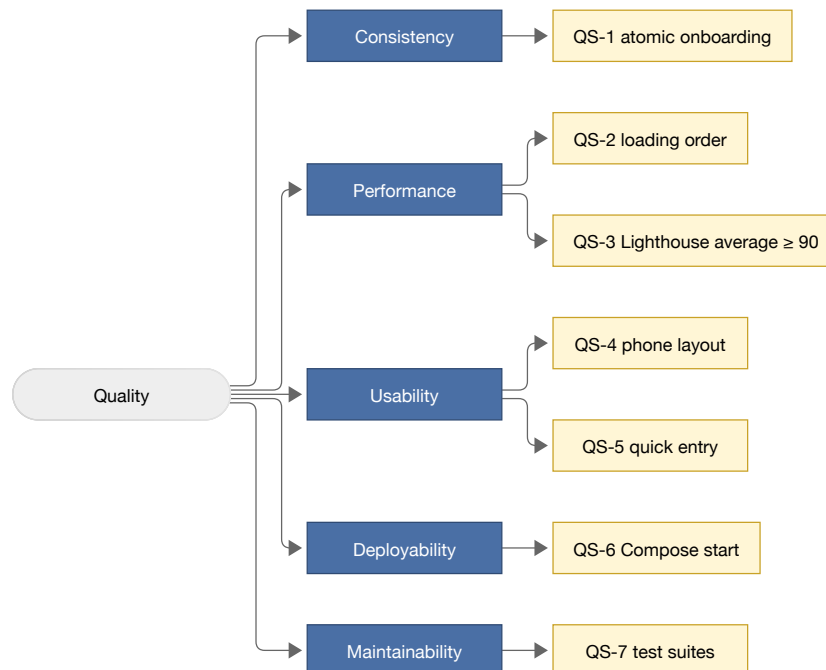


Figure 14: QS-1, QS-2, QS-4, and QS-5 check the quality goals of ch. 1; QS-3, QS-6, and QS-7 check the lecturer's requirements.

10.2. Quality Scenarios

Quality	Steps to reproduce	Expected outcome
Consistency		
QS-1 Atomic onboarding (goal 1)	1. Sign in as <code>demo@finance.local</code> in local development. 2. In <code>backend/local.db</code>, add a trigger that aborts inserts into <code>finance_account</code> (<code>raise(abort, ...)</code>). 3. Finish the onboarding wizard, then drop the trigger.	Submit fails with 500. No household, household_member, or category row exists for the user; the wizard draft is still in <code>localStorage</code> .
Performance		
QS-2 Loading order (goal 2)	1. Sign in at <code>http://localhost:8080</code>. 2. DevTools → Network: disable cache, throttle to "Slow 4G". 3. Reload <code>/transactions</code>.	Shell and transactions chunks start before <code>get-session</code> and <code>households/me</code> return. Other route chunks start 3 s after the first navigation completes.

Quality	Steps to reproduce	Expected outcome
QS-3 Lighthouse	1. <code>docker compose up --build</code>, sign in as <code>sample@finance.local</code>. 2. DevTools → Lighthouse, all four categories, on Mobile and Desktop for each shell tab.	The four category scores average 90 or more, on Mobile and on Desktop; a single category may be lower.
Usability		
QS-4 Phone layout (goal 2)	1. Open <code>/transactions</code> on a phone (iOS Firefox) or in the Playwright <code>mobile</code> project. 2. Scroll to the end of the list.	Bottom navigation stays visible; the document does not scroll, only the tab panel does.
QS-5 Quick entry (goal 3)	1. On a phone, add one expense with account and category. 2. Tap <code>+</code>, type an amount, tap <code>Save</code>.	Keyboard is open when the dialog appears; account and category match the previous entry; the entry saves without touching another field.
Deployability		
QS-6 Compose start (M20, ADR-6)	1. Fresh clone: <code>cp .env.example .env</code>, set <code>BETTER_AUTH_SECRET</code>. 2. <code>docker compose up --build</code>. 3. Sign in at <code>http://localhost:8080</code> as <code>demo@finance.local</code> / <code>demo-password</code>, enter data. 4. <code>docker compose restart</code>.	Both containers report healthy; the demo user reaches onboarding; data entered before the restart is still there.
Maintainability		
QS-7 Test suites	1. Backend: <code>bun run test</code>, <code>bun run test:e2e</code>. 2. Frontend: <code>bun run test</code>, <code>bun run e2e</code>.	All four suites pass (scope and counts in ch. 8 Testing).

10.3. Lighthouse Results

Production bundle via `docker compose` (port 8080), 2026-09-25. The three recorded runs meet QS-3; other tab and device combinations were not recorded.

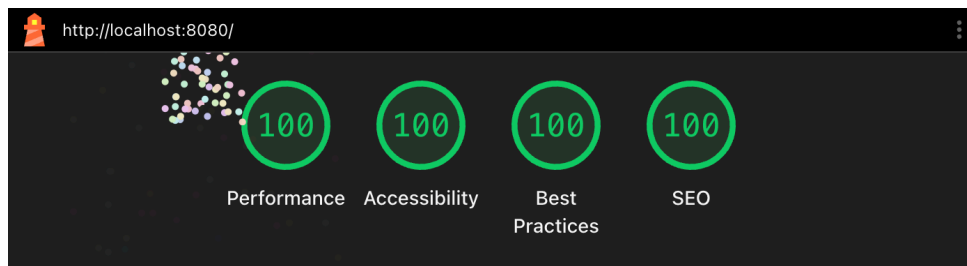


Figure 15: Home /, Desktop: average 100.

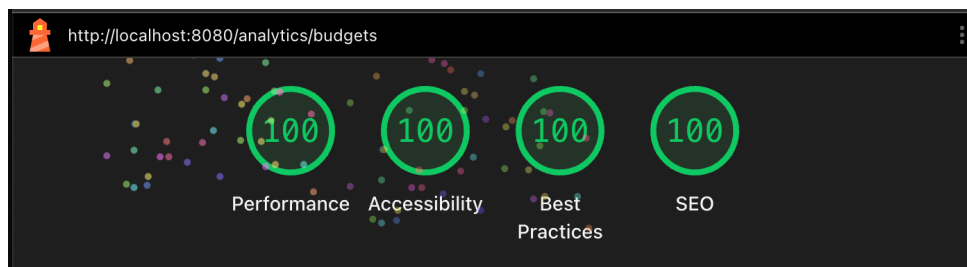


Figure 16: Budgets /analytics/budgets, Desktop: average 100.

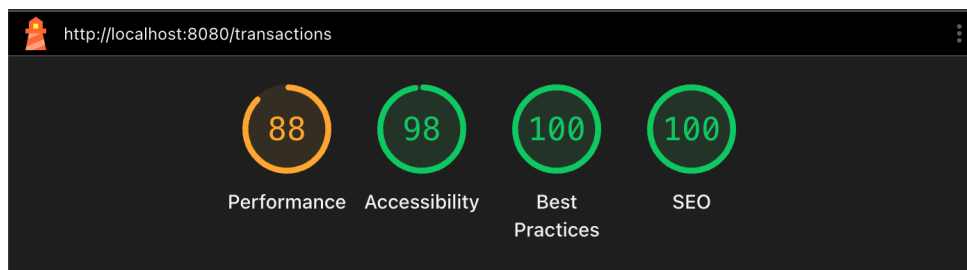


Figure 17: Transactions /transactions, Mobile: average 96.5 (Performance 88, Accessibility 98).

11. Risks and Technical Debt

Risk / debt	Impact	Mitigation
No TLS in the Compose stack	nginx serves plain HTTP on 8080; outside localhost, credentials and the session cookie travel unencrypted.	Terminate TLS in front of nginx (with trusted real-IP, see proxy row) and set BETTER_AUTH_URL/TRUSTED_ORIGINS to the https origin.
No fault-injection test for onboarding and category transfer	A regression to sequential writes there would pass all suites; only the manual check QS-1 covers onboarding.	Add trigger-based tests like the budget ones in <code>budget.e2e-spec.ts</code> .
SQLite single writer, one API instance	No horizontal scaling; concurrent writes serialize.	Adequate for household load; switching driver revisits ADR-3.
Proxy trust boundary	An extra proxy in front of nginx makes all clients share one rate-limit key.	Configure trusted real-IP in nginx before adding a proxy (ch. 8 Security).
Client clock drives auto take-over	A wrong clock may fill a month early.	User removes copies; see ADR-2.
better-auth CLI schema generation	Generated <code>auth.schema.ts</code> can lag the library version.	Review generated diff after <code>bun run auth:generate</code> .

12. Glossary

Term	Definition
Household	Ownership root of all financial data; has one base currency.
Household member	User who belongs to a household, as owner or member; the end user.
Finance account	A money container (checking, savings, cash, ...) with opening balance; table <code>finance_account</code> .
Transaction	Positive amount of type <code>expense</code> or <code>income</code> booked on one finance account.
Uncategorized	Bucket of transactions without category (<code>category_id</code> null), ADR-1.
Limit (budget)	Spending cap of one category for one month <code>YYYY-MM</code> ; no row = no limit.
Take-over	Copying the nearest earlier month's limits into an empty month, ADR-2.
Touched month	Month recorded in <code>budget_month</code> ; never auto-filled again.
Minor units	Integer cents used for every stored amount.
Shell	Authenticated layout with tab navigation and the single scroll panel.